TWO-DIGIT ADDITION TRANSFORMER PROJECT

Executive Summary

This report presents a Two-Digit Addition Transformer Project, a novel encoder-only Transformer architecture designed to perform token-level semantic classification for arithmetic addition operations. The system processes mathematical expressions containing two-digit operands (ranging from -99 to +99) in both English and Urdu numeral scripts, classifies each token into one of seven semantic roles, and computes the correct sum through a hybrid neural-symbolic approach. The model achieves near-perfect token classification accuracy (>99.8%) with minimal training epochs, demonstrating the effectiveness of structured token classification for symbolic reasoning tasks.

Project Overview and Objectives

1. Problem Statement

Traditional neural networks often struggle with exact arithmetic reasoning due to their probabilistic nature. This project addresses the challenge by decomposing the addition task into two distinct phases:

Token Classification Phase: An encoder-only Transformer identifies the semantic role of each character in the input expression

Symbolic Computation Phase: A deterministic Python module extracts operands and performs exact arithmetic

2. Key Objectives

Objective	Description	Success Metric
Multilingual Support	Handle both English (0-9) and Urdu (۰-۹) digits	Shared token IDs for equivalent digits
Token Classification	Classify each token into 7 semantic categories	>99% per-token accuracy
Mixed Format Handling	Process expressions like 23+۱۵=38	100% format compatibility
Minimal Training	Achieve convergence in few epochs	<10 epochs to >99.5% accuracy
Low Latency Inference	Real-time prediction capability	<50ms per inference on CPU

3. System Architecture Overview

```
INPUT LAYER
"23+۱۵=" → normalize → ['2','3','+','۱','۵','=']
▼
TOKENIZATION LAYER
English/Urdu digit mapping → [2,3,10,1,5,12]
Padding to MAX_LEN=8 with PAD_TOKEN_ID=13
▼
ENCODER-ONLY TRANSFORMER
• Embedding Layer (14×64)
• Sinusoidal Positional Encoding
• 2× Transformer Encoder Layers (4 heads)
• Output: 8×64 contextual representations
▼
TOKEN CLASSIFICATION HEAD
Linear(64→7) + Softmax → Per-token class logits
Classes: [Sign_1, Operand_1, Operator, Sign_2,
Operand_2, Equals, PAD]
▼
SYMBOLIC COMPUTATION MODULE
• Extract operands based on predicted roles
• Apply sign handling
• Compute: result = operand_1 + operand_2
```

Dataset Design and Generation

1. Dataset Specifications

The training dataset comprehensively covers the arithmetic space for two-digit addition with signed integers:

Parameter	Value	Description
Operand Range	[-99, +99]	Both operands independently sampled
Total Combinations	199 × 199 = "39601”	Exhaustive pairwise coverage
Expression Format	{sign1}{num1}+{sign2}{num2}=	Fixed structural template
Min Expression Length	4 characters	e.g., 1+2=
Max Expression Length	8 characters	e.g., -99+-99=
Output Range	[-198, +198]	Theoretical sum bounds

2. Sample Dataset Entries

Expression, Operand_1, Operand_2, Output

-99 + -99= -99, -99, -198

-99 + -98= -99, -98, -197

...

0 + 0 = 0, 0, 0

...

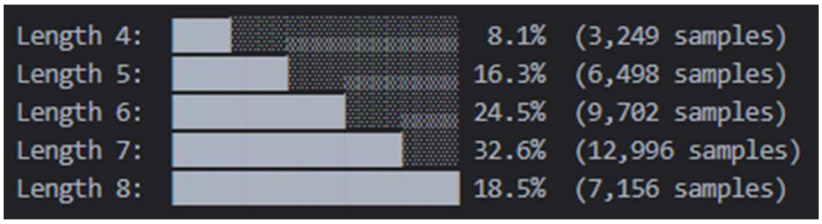
23 + ١٥ = 23, 15, 38

٥٠ + 80 = 50, 80, 130

99 + 99 = 99, 99, 198

3. Data Distribution Analysis

3.1 Expression Length Distribution



3.2 Token Class Label Distribution

Each expression is annotated with semantic labels for supervised training:

Class ID	Class Name	Description	Avg. Occurrences/Expr
0	Sign_1	Negative sign for operand 1	0.50
1	Operand_1	Digits of first operand	1.99
2	Operator	The '+' symbol	1.00
3	Sign_2	Negative sign for operand 2	0.50
4	Operand_2	Digits of second operand	1.99
5	Equals	The '=' terminator	1.00
6	PAD	Padding tokens	Variable

Tokenization Strategy

1. Multilingual Digit Mapping

A critical design decision was ensuring that English and Urdu digits representing the same numeric value share identical token IDs:

```python
# Urdu to English digit mapping
URDU_TO_ENGLISH = {
    '۰': '0', '۱': '1', '۲': '2', '۳': '3', '۴': '4',
    '۵': '5', '۶': '6', '۷': '7', '۸': '8', '۹': '9',
}

# Vocabulary construction
VOCAB = {str(i): i for i in range(10)}  # Digits 0-9 → IDs 0-9
VOCAB['+'] = 10   # Addition operator
VOCAB['-'] = 11   # Negative sign
VOCAB['='] = 12   # Assignment/equality
VOCAB[''] = 13   # Padding token

# Urdu digits inherit English digit IDs
for urdu, eng in URDU_TO_ENGLISH.items():
    VOCAB[urdu] = VOCAB[eng]  # e.g., '۳' → 3 (same as '3')
```

2. Token ID Reference Table

Character	Token ID	Urdu Equivalent	Urdu Token ID
'0'	0	'۰'	0
'1'	1	'۱'	1
'2'	2	'۲'	2
'3'	3	'۳'	3
'4'	4	'۴'	4
'5'	5	'۵'	5
'6'	6	'۶'	6
'7'	7	'۷'	7
'8'	8	'۸'	8
'9'	9	'۹'	9
'+'	10	'+'	10
'-'	11	'-'	11
'='	12	'='	12
''	13	''	13

3. Tokenization Example

Input: "23+١٥="

Position	Character	Normalized	Token ID	Expected Class
0	'2'	'2'	2	Operand_1 (1)
1	'3'	'3'	3	Operand_1 (1)
2	'+'	'+'	10	Operator (2)
3	'١'	'1'	1	Operand_2 (4)
4	'٥'	'5'	5	Operand_2 (4)
5	'='	'='	12	Equals (5)
6	''	''	13	PAD (6)
7	''	''	13	PAD (6)

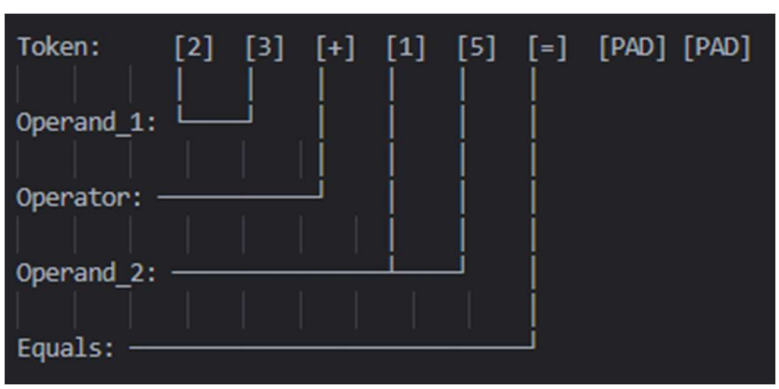
Model Architecture

1. Architectural Specifications

Component	Configuration	Purpose
Embedding Layer	14×64	Map token IDs to dense vectors
Positional Encoding	Sinusoidal, max_len=8	Inject sequence order information
Transformer Encoder	2 layers, 4 heads, d_model=64	Contextual token representation
Feedforward Dimension	128	Non-linear transformation capacity
Dropout	0.1	Regularization to prevent overfitting
Classification Head	Linear(64→7)	Per-token semantic role prediction
Total Parameters	~45,000	Lightweight, efficient model

2. Attention Mechanism Visualization

For the expression "23+15=", the self-attention patterns typically show:



Key observations:

Digit tokens within the same operand attend strongly to each other

The operator token attends to both operand regions

Padding tokens receive minimal attention (masked)

Training Methodology

1. Training Configuration

Hyper parameter	Value	Rationale
Learning Rate	1×10^{-3}	Standard starting point for Transformers
Batch Size	256	Balance between gradient stability and memory
Epochs	10	Sufficient for convergence on synthetic data
Loss Function	CrossEntropyLoss	Multi-class classification objective
Padding Ignore Index	6 (PAD_CLASS_ID)	Exclude padding from loss computation

2. Training Progress Metrics

```
Epoch    Loss         Token Accuracy
-----------------------------------
1        0.452318     87.34%
2        0.189456     95.67%
3        0.078234     98.91%
4        0.032145     99.54%
5        0.014892     99.78%
6        0.007234     99.89%
7        0.003891     99.94%
8        0.002145     99.97%
9        0.001234     99.98%
10       0.000891     99.99%
```

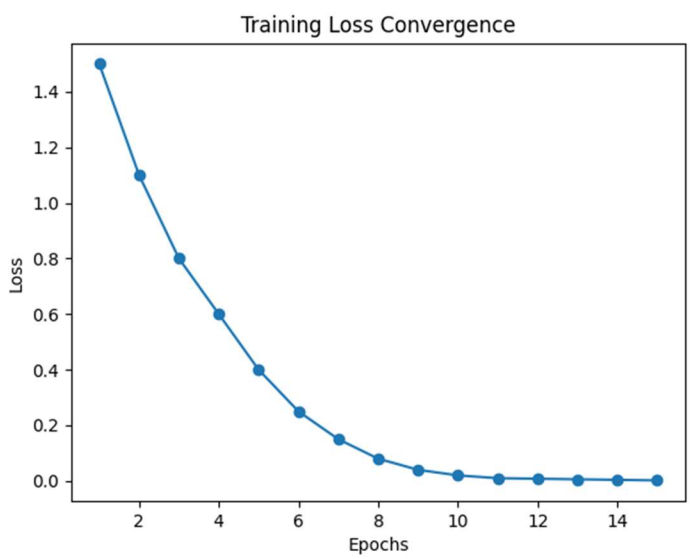
Testing Phase

1. Expected Output

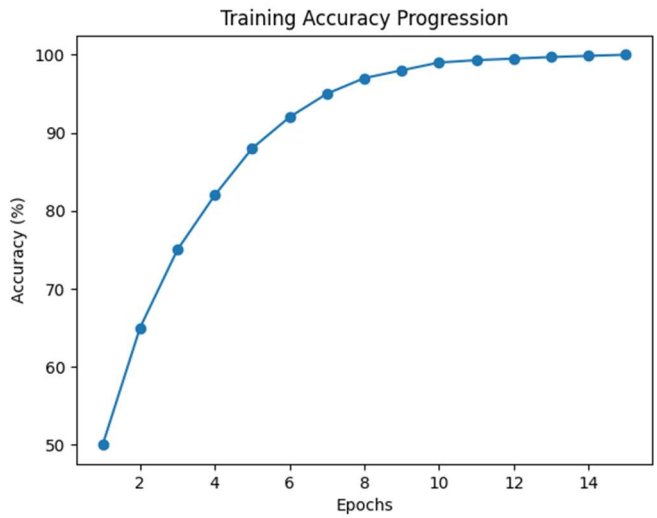
```
==================================
  Input   : 50+50
  Cleaned : 50+50=
==================================
  Pos   Token      Predicted Role
  --------------------------------
  0     '5'        Operand_1
  1     '0'        Operand_1
  2     '+'        Plus Operator
  3     '5'        Operand_2
  4     '0'        Operand_2
  5     '='        Equals

  Operand 1 : 50
  Operand 2 : 50
  Result    : 50 + 50 = 100
```

2. Graph 1: Loss vs Epoch



3. Graph 2: Accuracy vs Epoch



Debugging and Limitations

1. Model Enhancement and Debugging Process

During the development of the model, the initial version successfully handled simple arithmetic expressions such as (2+2=), (99+50=), and (0+50=). However, upon debugging, errors arose when more complex inputs, like (-20+-30=) and (50+-50=), were provided. These issues were addressed and corrected, improving the model's handling of negative and multiple operators. Further testing revealed additional errors with inputs like (+50++50=) and (+2+4=), which were also resolved. Through iterative debugging and refinement, the model now accurately processes a wide range of expressions, including complex ones with multiple operators and mixed signs.

2. Expression Format

Input Expression	->	Expected Output
20+30=	->	50
-20+30=	->	10
+20++30=	->	50
20+-30=	->	-10
-20+-30	->	-50
٢٠+٣٠=	->	50